

交代工作，然後學會驗收

Prompt Engineering 不是研究神奇咒語，
而是把腦中的需求，轉成能執行、
也能驗收的工作說明。



CHANG JUI-MING

—— 張睿明 / AIMing Data



本堂路線

01

把工作交代清楚

四個必答問題、CO-STAR，以及什麼時候不必填完整張表。

02

好的 Prompt 是改出來的

觀察落差、修改、再試一次。
由講者示範四輪 Prompt 的變化。

03

Prompt 只是其中一層

Agent $\approx$ Model + Harness，問題不一定出在模在模型。

文章貫穿主要示範，搭配三支影片；最後用雨量分析，預告工作流與資訊交接。



本週 AI 焦點

2026/9/28–10/2

官方公告與延伸閱讀

[10/1](#)

[Claude 官方](#)

Claude Code mods

用擴充功能客製介面與流程，讓 AI 配合你的工作方式。

[9/29](#)

[OpenAI 官方](#)

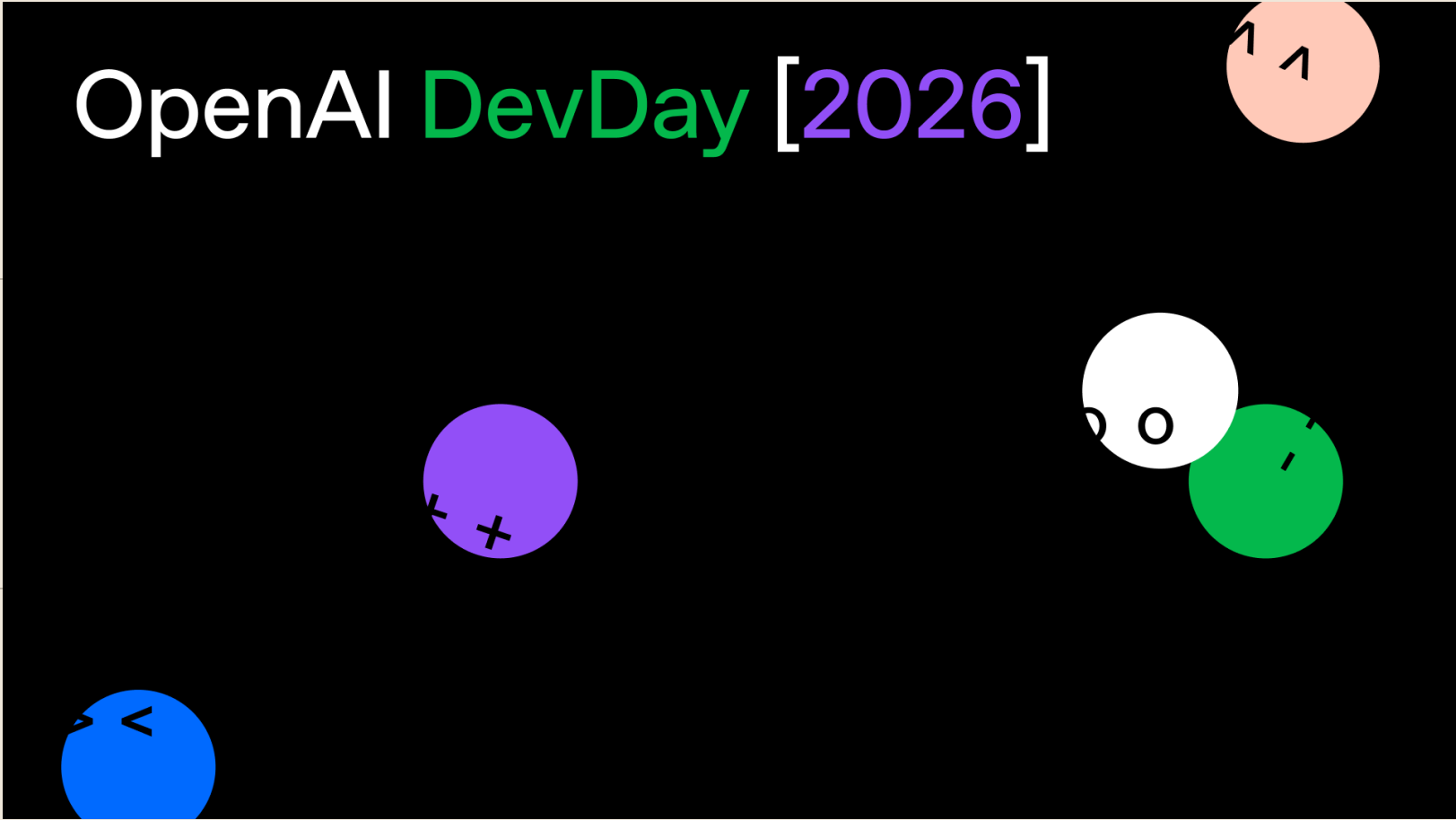
OpenAI dots

AI 助理持續推進任務；目標、限制與驗收標準要說清楚。

Karpathy 觀點：輸出格式影響理解

[數位時代文章](#)

依閱讀對象與用途，選擇清楚文字、圖解、網頁或影片。



任務、背景、限制、輸出格式，都要交代清楚



01

怎麼用自然語言 把工作交代清楚

AI 不會因為我們心裡想得很清楚，就自動通靈你的腦袋。



都是中文，但都沒說清楚

幫我整理這篇文章。

是摘要、分類、改寫，還是做成表格？

幫我把這篇文章改寫一下。

給熟悉 AI 的人，還是第一次學 Prompt 的人？

幫我把這篇文章做成簡報大綱。

要講主旨、比較四版 Prompt，還是教操作步驟？

AI 不會因為我們心裡想得很清楚，就自動通靈你的腦袋。



常見的「咒語」

你是一位世界頂尖專家

請深呼吸

一步一步思考

如果答得好，我會給你小費

看久了會懷疑：使用 AI 是在交代工作，還是在召喚儀式？

把腦中的需求，轉成 AI 可以理解、執行與驗收的工作說明。

Anthropic 把 Claude 類比成新承包商：
它不知道你的背景與任務，需要具體資訊。

「幫我弄一下那個東西。」

等等，「那個東西」從頭到尾都沒有正式出場。



先檢查四類資訊，再決定要補什麼

01

情境

現在是什麼背景？這件事為什麼要做？

02

任務

要完成什麼？動詞要具體。

03

限制

不能做什麼？怎麼判斷好壞？

04

交付

結果要長成什麼樣子？

Prompt 的價值，往往不是讓模型突然變聰明，而是**減少它需要自由發揮的空間**。



[共同文章：Day 09 | 好的 Prompt 為什麼需要迭代？](#)

原本

幫我整理這篇文章。

AI 得自己猜：讀者懂多少、哪些內容不能亂加、最後要長成什麼樣子。

右邊那一版沒有更華麗的詞，只是把任務邊界說清楚。

背景

我要導讀〈好的 Prompt 為什麼需要迭代？〉。
讀者用過 ChatGPT，但第一次學 Prompt。

任務

整理文章主旨、作者例子、操作步驟與適用限制。

限制

- 只依據我附上的文章，不補寫原文沒有的主張
- 專有名詞第一次出現時，附一句白話解釋
- 無法確認的內容，標記「原文未說明」

輸出格式

使用 Markdown，依四個標題輸出：
主旨 / 作者例子 / 操作步驟 / 適用限制

先附上文章內容；網址不等於 AI 已讀全文。摘要仍要回到原文核對。



CO-STAR：一張交代工作的檢查表

C**Context**

什麼背景下發生？

O**Objective**

要完成什麼任務？

S**Style**

用什麼寫作方式？

T**Tone**

呈現什麼態度？

A**Audience**

寫給誰看？

R**Response**

最後怎麼輸出？

它不能保證滿分答案，卻能提醒我們是否漏講條件。像旅行前的清單：去便利商店不用檢查護照；出國一個月時，最好也別只記得帶充電線。

背景：GovTech 於 2023 年 11 月舉辦首屆 GPT-4 提示工程競賽。



原本

把這篇文章改得好懂一點。

「好懂」要對誰而言？
沒有讀者與用途，AI 只能自己猜。

最容易漏掉的是 Audience。
同一篇文章，給初學者與熟悉 AI 的人，
需要保留的解釋不一樣。

Context

用同一篇 Day 09 文章，準備線上 AI 入門導讀。

Objective

解釋為什麼需要迭代，以及如何觀察與修正落差。

Style

先說主旨，再用原文的四版影片 Prompt 說明。

Tone

親切、清楚、短句，不用浮誇宣傳語。

Audience

用過 ChatGPT，但第一次學 Prompt 的大學生。

Response

500 字內的 Markdown：主旨 / 作者例子 / 操作步驟。
只依據附文；無法確認的內容寫「原文未說明」。



同一篇文章，三種問法

A · 只有任務

「幫我整理這篇文章。」

觀察：長度、詳略、格式都由它決定。

B · 加上交付

「整理成五點摘要，
每點一句，使用繁體中文。」

觀察：形狀固定了，但內容取捨仍是它決定的。

C · 加上受眾與限制

「給第一次學 Prompt 的大學生：
整理主旨、作者例子與操作步驟，
共五點，每點一句，繁體中文。
只依原文；無法確認就標記
『原文未說明』。」

觀察：有了取捨與驗收的依據。

講者依序比較：任務 → 交付 → 受眾與限制；不用先猜哪個答案最好。



不是每件事都要填完整張表

簡單、低風險

請把附上的 Day 09 文章整理成五點摘要，使用繁體中文。

交代任務與輸出就夠了。

有特定讀者或風格

把同一篇文章改寫成初學者導讀。讀者用過 ChatGPT，但第一次學 Prompt。親切、清楚，500 字內，只依原文。

補上受眾、用途與語氣。

複雜、會重複、要交接

用 CO-STAR 或其他結構化框架，把條件整個攤開。

要學的不是每次都寫 CO-STAR，而是知道何時該補哪些資訊。

Style 是文章怎麼寫（科普、新聞稿、操作手冊）；Tone 是給人的感覺（正式、親切、謹慎）。
一篇文章可以是科普風格，同時保持親切語氣。



同一篇文章，交付格式跟著用途走

以下是講者整理的教學示意；格式換了，文章依據不能跟著變。

自己閱讀、之後還要改

Markdown 摘要

比較四版 Prompt

表格

交給程式處理

JSON

向初學者導讀

簡報大綱

主旨：觀察落差，再修改 Prompt

- 視角

- 音樂與畫面氛圍

```
{
  "main_point": "觀察落差，再修改 Prompt",
  "checks": ["視角", "音樂與畫面氛圍"]
}
```

Markdown 便於閱讀；JSON 便於程式處理，內容仍需驗證。



不知道怎麼寫，就先把需求說給 AI 聽

先說清楚想法，再請 AI 整理 Prompt。
可以留在原對話繼續；若另開對話，
要把 Prompt 和原文一起提供。

讓 AI 先提問：讀者知道什麼？
要摘要、比較表，還是導讀大綱？
哪些內容只能依原文？

AI 也可以協助把問題問完整。

我想請 AI 完成下面的工作，
但還不知道如何寫成清楚的 Prompt。

請先不要執行任務：

1. 找出我缺少哪些必要資訊
2. 最多詢問我五個問題
3. 根據回答，用 CO-STAR 整理成 Prompt
4. 將最終 Prompt 放在程式碼區塊中

我的初步需求是：

把附上的 Day 09 文章，整理成
給 AI 初學者聽的十分鐘導讀大綱。

Demo: GPT-Live



換上制服，不會自動增加專業知識

只有帽子

你是一位專業文章編輯。

角色能幫模型選擇比較適合的詞彙與觀點，但不會替它增加最新知識。

帽子 + 可檢查的規則

你是一位協助初學者閱讀 AI 文章的教學助理。

請整理附文的主旨、作者例子與操作步驟。
只依據原文；無法確認就寫「原文未說明」。
第一次出現的專有名詞，附一句白話解釋。
最後檢查每一點是否能在原文找到依據。

真正有用的不是「教學助理」這頂帽子，而是後面可執行、可檢查的規則。制服可以幫助進入角色，但不能代替職前訓練。



一個好的 Prompt，不等於一定有正確答案

誤解資料或忽略部分限制

使用過時資訊

格式正確，但內容不正確

補出不存在的內容

產生看似合理的錯誤

把 Prompt 想成工作單：

寫得好，師傅比較不容易跑錯地方；

但完工後，屋主還是要驗收。

不能因為地址與油漆色號都寫得漂亮，就假設牆面沒有漏刷。高風險領域中，清楚的 Prompt 只是起點，不是最後簽核。



講者示範：把需求補完整

「幫我整理這篇文章。」

請把 Day 09 文章整理成五點摘要，
給第一次學 Prompt 的大學生。只依原文，
每點一句，使用繁體中文。

課後可換一篇文章，再用這四題檢查自己的 Prompt。

先問自己這四題

「整理」是要做什麼？

整理文章的主旨、
例子與操作步驟。

誰要用這個結果？

第一次學 Prompt
的大學生。

哪些不能動？

不能補寫原文
沒有的主張。

要交回什麼？

五點摘要，每點一句，
使用繁體中文。



換個框架， 仍然交代同一份工作

RCTCO

把角色、背景與交付拆開看

Role	AI 入門課的教學助理
Context	用 Day 09 文章準備導讀
Task	整理主旨、例子與操作步驟
Constraints	只依原文；不明處要標記
Output	五點摘要， 每點一句， 繁體中文

框架整理：數位時代原圖；示範任務統一改用 Day 09 文章。

RACE

先說要做什麼，再給示例

Role	協助初學者閱讀的編輯
Action	將 Day 09 文章改寫成導讀
Context	讀者用過 ChatGPT，第一次學 Prompt
Example	主旨：觀察落差，再修改 Prompt。 例子：四版影片逐步補上條件。
檢查	示例的內容也要有原文依據



任務複雜度不同， 補充的細節也不同

RISEN

把步驟與驗收條件說清楚

Role	AI 入門課的教學助理
Instructions	整理 Day 09 文章，協助初學者導讀
Steps	先抓主旨，再整理例子與操作步驟
End goal	讀者能說出如何觀察與修正落差
Narrowing	500 字內，只依原文，保留限制

RTF

簡單任務，三個條件就能開始

Role	文章導讀助理
Task	整理 Day 09 的四版 Prompt 差異
Format	表格：版本 / 新增條件 / 觀察落差
驗收	四版內容正確，沒有添加新情節
提醒	缺的資訊再補，不必每次套滿框架

框架整理：數位時代原圖；共同原則仍是情境、任務、限制與交付。



02

好的 Prompt 是改出來的

我說的「合理」，和模型認為的「合理」，不一定是同一個合理。

[播放影片 · 四版 Prompt \(0:47\)](#)

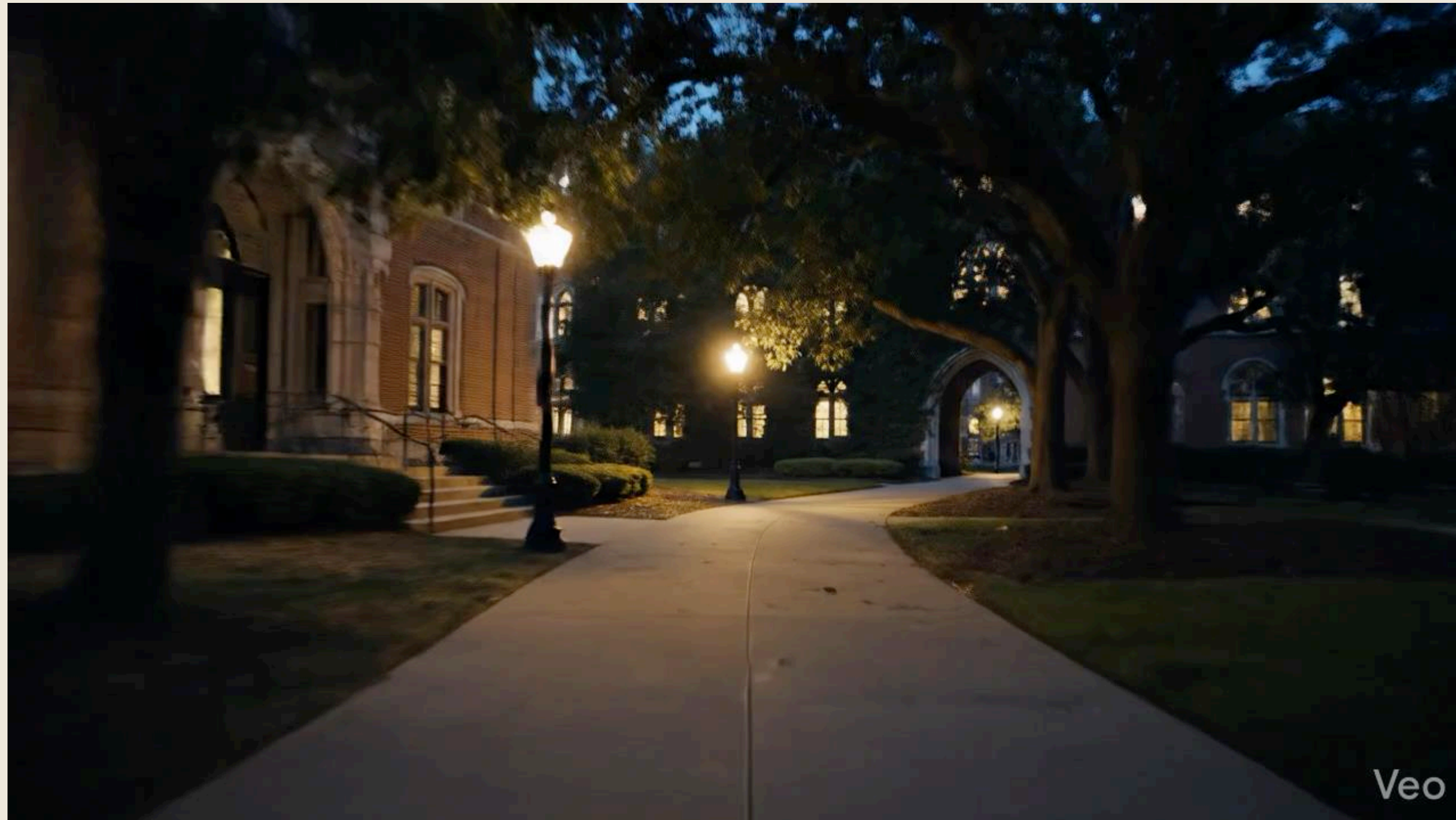


Vol. 1



**一個人漫步在
夜晚的校園，
然後出現怪物**

Vol. 2



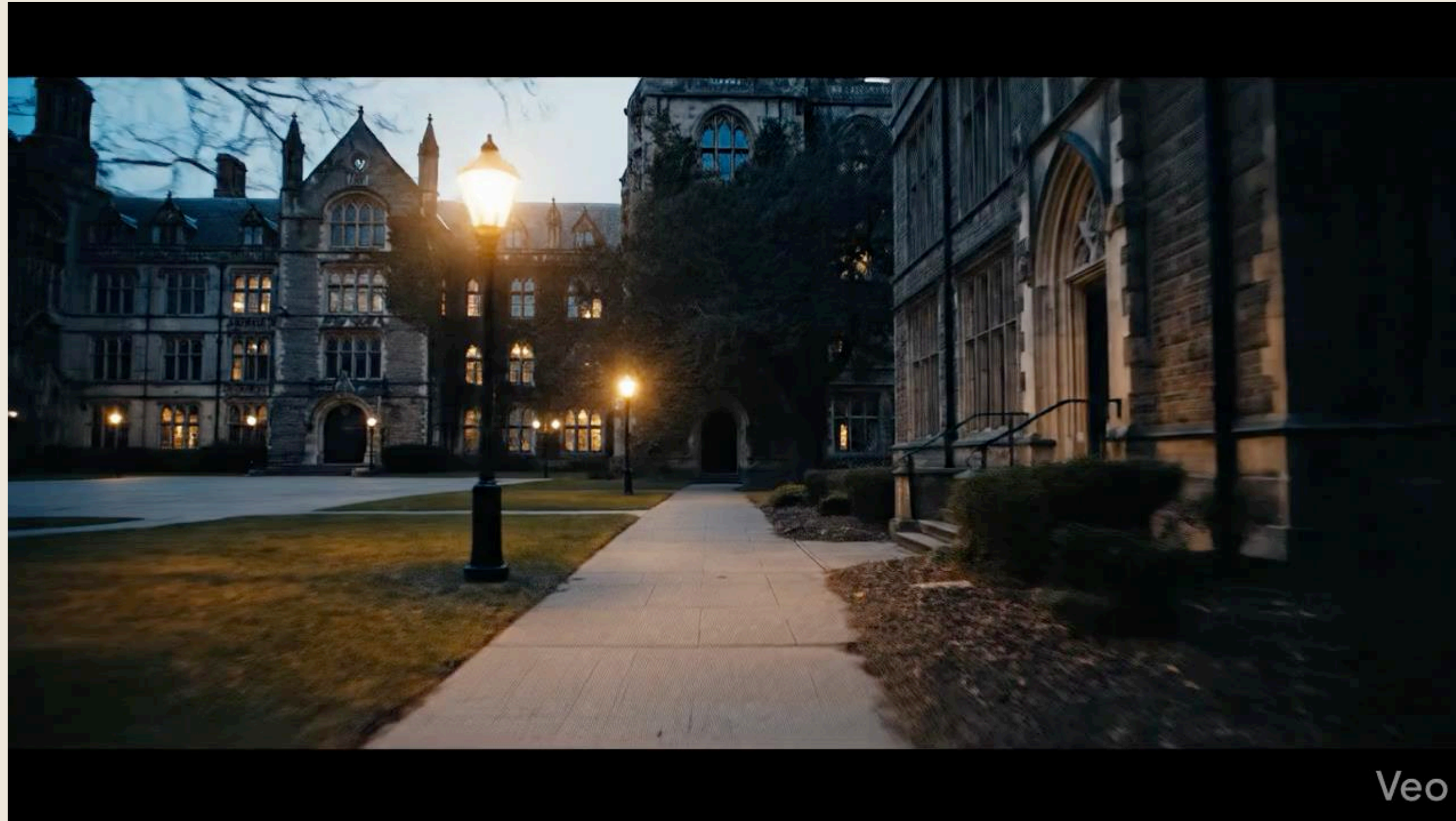
用第一人稱視
角，一個人漫
步在夜晚的校
園，然後突然
出現怪物

Vol. 3



用第一人稱視角，
一個人漫步在夜
晚的校園，然後
突然出現怪物，
要有驚悚的感覺！

Vol. 4



用第一人稱視角，
一個人在夜晚的
校漫步園，然後
突然出現怪物，
畫面和音樂都要
驚悚的感覺！

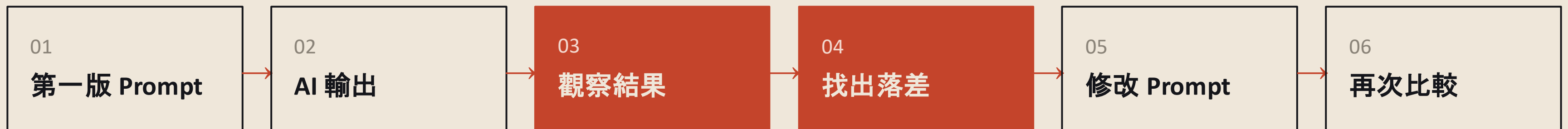
同一個畫面， 四版 Prompt

v1	一個人漫步在校園裡， 後來出現一隻怪物。	模型對這句話有自己非常獨特的見解
v2	用第一人稱視角， 一個人漫步在校園裡， 後來出現一隻怪物。	視角對了， 但結尾還是歪樓
v3	用第一人稱視角， 一個人漫步在校園裡， 突然出現一隻怪物， 要有驚悚的感覺！	音樂有氛圍了， 畫面停在奇妙的拐點
v4	用第一人稱視角， 一個人漫步在校園裡， 突然出現一隻怪物， 音樂和畫面要有驚悚的感覺！	逐漸趨近一開始心中的畫面

影片檢查視角與氛圍；回到文章， 我們檢查重點、原文依據與格式。



先定驗收標準，再開始 Prompt 迭代



真正困難的不是這個流程，而是：你看不看得出來哪裡需要改？

所以 Prompt Engineering 的核心能力，不只是會寫 Prompt，更是會驗收。





把腦袋裡的流程搬出來

「做一片果醬吐司」看似簡單，
省略步驟卻容易讓對方誤解。
跟 AI 對話，也要把流程拆清楚。

回到文章：讀原文 → 抓主旨 → 摘要 → 核對依據。

從哪裡開始導入？

高頻率

高重複

低決策風險

先找自己常做、反覆做、
而且出錯時容易檢查與修正的工作。

複雜任務牽涉工具與流程時，
不能只靠修改 Prompt 解決。



文章摘要怎麼驗收？從一個錯例開始

步驟	同一篇文章的教學示意	驗收重點
原有依據	四版逐步補上視角、驚悚感，以及音樂與畫面。	對照第 21 頁
有問題的摘要	「加上第一人稱，就能保證影片符合期待。」	「保證」超出依據
指出落差	第二版之後仍在修正，不能說一次就保證成功。	留下可核對的描述
給一句回饋	刪除「保證」，只整理四版逐步補上的條件。	不加例子外的結論
改後摘要	觀察落差，逐步補上第一人稱及音樂、畫面氛圍。	主旨與依據一致
再次核對	視角、音樂、畫面均能回指四版例子；無成功保證。	改後也要查

先看標準：保留主旨、有依據、不加保證。以上為講者示意，非實測輸出。



人的工作開始往上移

以前要自己畫圖、自己寫程式、自己做簡報。現在 AI 可以協助完成很多執行工作，但人開始需要負責另一層。

有些工作確實降低了技術門檻，但驗收門檻不一定跟著消失——有時因為 AI 做得太像真的，驗收反而更重要。

定義問題

描述需求

判斷成果

發現偏差

提供回饋

決定下一輪怎麼改



你要先知道「好」長什麼樣子

「把這篇文章的摘要做得更好。」



字變少、語氣更漂亮，
就代表主旨整理得更準嗎？

不知道。因為「好」沒有被定義。

把「好」寫成可檢查的條件：

初學者看得懂；保留文章主旨；每一點都有原文依據；

沒有自行添加作者未說的結論。

每輪修正，讓需求表達與這次成果更接近目標。



03

Prompt 只是 其中一層

模型像一顆引擎，外面還要接上
方向盤、煞車、儀表板與安全帶。



Agent 可以先想成 Model + Harness

Agent：在目標下選擇下一步、使用工具，再依結果調整。

MODEL

引擎與輸入

模型依收到的資訊理解、生成與判斷。
Prompt：文章摘要的要求。
Context：本次提供的文章、指令與歷史。

HARNESS

方向盤、煞車、儀表板

讀入文章、協調工具、保存進度，
再把結果接回工作流程。

- Function Calling
- MC
P
- Memory
- Context management
- Skills

這是理解用的類比；以上名詞留待後續課，不必一次學完。



Agent 表現不好， 問題不一定出在模型

Prompt 寫得不清楚

Context 塞錯資料

Harness 的流程設計有問題

也要檢查模型是否適合任務

「換一顆更大的模型看看。」

有點像車子方向盤鬆掉，處理方式卻是換一顆更大的引擎。

馬力增加了。方向盤還是鬆的。



可調整的寫作規則， 要用結果來驗證

規則不是越多越好。針對可調整的寫作方式，用同一組例子比較，再決定是否刪減。原文依據與任務必要條件仍要保留。

01

挑一條可測試規則

02

用同組例子比較

03

品質不退步才刪除

例如：比較有沒有「先列三個小標題」，摘要是否同樣清楚。
「只依原文」等必要條件要保留；一次改一項，再重複檢查。



真正的系統， 不是在找一個完美 Prompt

Anthropic 談 Context Engineering 時也指出：Agent 的行為不只受 Prompt 影響， 還包含工具、MCP、外部資料、訊息歷史， 以及 Context 管理。

而不是每天研究：「請」改成「務必」會不會讓準確率多 3%？

Prompt Engineering 是很好的入口， 因為它讓我們第一次思考：人要如何把意圖交給模型？

可能需要調整的幾層

Prompt
Context
Tools
Memory
Workflow
Guardrails
Evaluation
Model

這是後續課的地圖， 非必備清單。



「請 AI 分析雨量資料」之後，還有哪些決定？

情境假設：手上有一份雨量檔，想請 AI 找出值得注意的變化。

01	02	03	04
目的與資料	→ 程式與工具	→ 核對與重跑	→ 圖表或報告
先定問題與使用對象； 檢查欄位、單位與缺測， 再選方法與所需資料。	請 AI 寫分析程式， 由工具讀檔、執行計算。 不能只靠 AI 看一眼數字。	核對計算、單位與缺測處理， 確認結果是否合理； 有問題，就回前一步修改。	依前面定好的讀者與用途， 交付圖表、報告或兩者。 說明發現、限制與依據。

路徑可以迭代：換方法、修程式、重畫圖，都要有檢查結果作依據。

教學假設：本頁未執行資料分析，也不展示任何實際分析結果。



如果要分工，工作與資訊怎麼交接？

先理解兩個概念

Subagent

可選的分工方式。
不是每個步驟都必須另外開一個。

Context window

一次能處理的上下文容量，
不等於永久記憶。重要資訊要明確保存。

雨量分析的「交接包」

- 01 目的、資料欄位、單位與時間範圍
- 02 已選方法與採用原因
- 03 程式、版本與執行方式
- 04 圖表／報告與輸出檔位置
- 05 檢查發現、限制與未解問題
- 06 下一步、完成標準與執行者

後面的課再接著看：誰決定下一步？資訊怎麼留下？工具如何真正執行？



Prompt 迭代不是「AI 沒做好，所以我再叫它一次」。

而是：我觀察它做了什麼，找出和目標的差距，
再把自己的需求說得更清楚。

OBSERVE

觀察



REFLECT

反思



ITERATE

迭代

真正讓成果變好的，往往不是某一句傳說中的神 Prompt，而是使用者有沒有能力看見差距。



交代工作前，先問自己六件事

我究竟想完成什麼？

為什麼要做這件事？

結果要給誰看？

哪些內容不能出錯？

我準備如何判斷成果好不好？

這份結果下一步要拿去做什麼？

當需求沒有說清楚時，對方只能猜。AI 只是誠實地把這件事放大。

REFERENCES

Teo, S. (2023). How I Won Singapore's GPT-4 Prompt Engineering Competition

Anthropic. Introduction to Prompt Design

Anthropic. Prompt generator · Prompt improver (2024)

Anthropic. Demystifying evals for AI agents

Anthropic. Effective context engineering for AI agents

OpenAI. The next evolution of the Agents SDK

OpenAI. A practical guide to building agents

數位時代（2026）。Claude Code 與驗證
（Boris Cherny 經驗轉述）

練習：day08_practice.ipynb

[共同文章：A-Ming, Day 09 | Prompt 迭代](#)

